ATLAS Master Flightline Clock — Actual Working Codebase Review

Review Date: July 3, 2026
Codebase Version: v1.6.0 (per git history)
Status: READ-ONLY ANALYSIS — No code modifications made

---

1. Current System Architecture

1.1 Module Structure & Dependencies

The system follows a clean **MVC-style separation** across two packages plus a top-level entry point:

```
main.py                          ← Entry point / application bootstrap
├── core/
│   ├── database.py              ← SQLite persistence (heats + settings)
│   ├── engine.py                ← State engine (1Hz tick, heat phase logic)
│   ├── models.py                ← Pydantic data models (HeatConfig, AtlasDisplayModel)
│   ├── timers.py                ← Standalone countdown timer (QTimer-based, 10Hz)
│   └── config.py                ← JSON-based hardware config (panels/scale)
├── gui/
│   ├── windows.py               ← Operator Console (3-tab UI: Live / Schedule / Set-
tings)
│   ├── led_simulator.py         ← Virtual LED matrix renderer (QPainter pixel art)
│   ├── themes.py                ← CSS stylesheet constants
│   └── fonts.py                 ← 5x7 bitmap font definitions (A-Z, 0-9, symbols)
└── data/
    └── atlas.db                 ← SQLite database (runtime-created)
```

Dependency graph:

```
main.py
  ├── core.database.AtlasDatabase
  ├── core.engine.AtlasEngine ──→ core.models.AtlasDisplayModel
  ├── gui.windows.OperatorWindow ──→ core.engine, core.models, gui.themes
  └── gui.led_simulator.AtlasLEDSimulator ──→ gui.fonts.AtlasBitmapFont
```

Key architectural decisions:
- **PySide6** (Qt6) for all GUI and event loop
- **Pydantic** for data models
- **SQLite** for persistence (heats + settings key/value store)
- **Signal/slot** architecture: engine emits `model_updated` and `brightness_changed` signals
- **Fusion style** forced for cross-platform consistency (bypasses macOS native tab rendering issues)

1.2 How Startup Currently Works (`main.py`)

The bootstrap sequence is **linear and straightforward** (45 lines total):

```
1. QApplication created with Fusion style
2. AtlasDatabase() initialized (creates data/atlas.db if needed)
3. AtlasEngine(db) created ├─ starts 1Hz tick immediately
4. DisplayConfig loaded from DB settings (width, height, scale, brightness)
5. OperatorWindow(engine) created and shown
6. AtlasLEDSimulator(config) created and shown
7. engine.model_updated ─→ led_sim.update_model (signal connected)
8. engine.brightness_changed ─→ led_sim.set_brightness (signal connected)
9. app.exec() ─┘ event loop runs
```

Important: There is **no auto-boot to clock mode**. The system launches straight into both windows (Operator Console + LED Simulator) with the LIVE tab selected. The user sees the operator panel immediately — there is no "start in clock-only mode" option.

1.3 Display Rendering (`led_simulator.py`)

The `AtlasLEDSimulator` is a **pure software renderer** — it emulates LED hardware entirely in QPainter:

- **Fixed-size QWidget** — dimensions set at construction time from `config.display_width × config.display_height × config.led_scale`
- **Dual layout modes:**
- **Full mode** (≥96px wide, e.g. 128×64): 6-row layout — time, heat info, open/close window, phase label, big timer (2× scale), status band
- **Compact mode** (<96px wide, e.g. 64×64): 6-row stacked layout with abbreviated labels, single-scale timer
- **Bitmap font rendering:** Each character is 5×7 pixels, drawn as individual ellipses (simulating round LEDs)
- **Brightness:** `_apply_brightness()` modulates RGB values by `config.brightness / 100.0`. Live updates via signal.
- **Note:** There is a **duplicate `_draw_char` method** (lines 137-148 and 168-179). The second definition (with brightness support) overwrites the first at runtime — so brightness works, but the first definition is dead code.

1.4 Current UI / Settings Capabilities (`windows.py`)

The `OperatorWindow` uses a **custom navigation bar** (not QTabWidget — native tabs were retired in v1.2.0 due to macOS rendering issues). Three "pages" via `QStackedWidget`:

Tab 1: LIVE

- Large status display showing heat number, direction, and timer countdown
- ABORT / CANCEL button

Tab 2: SCHEDULE

- Form to add/edit heats: Heat Number, Thermalling Direction, Track Opens/Closes/Ends times
- Schedule table with all heats (ID, Heat, Dir, Open, Close, Status)
- Click-to-edit, add new, delete functionality

Tab 3: SETTINGS

Currently has:
Setting	Widget	Persisted To
Competition Name	QLineEdit	DB `settings.competition_name`

Display Size	QComboBox (128×64 / 64×64)	DB `settings.display_width` / `display_height`
Sim LED Scale	QLineEdit	DB `settings.led_scale`
Display Intensity	QSlider (10-100%)	DB `settings.brightness`

System Controls (at bottom of Settings tab):
Button	Action	Implementation
RESTART ATLAS	`os.execv()` to relaunch process	`_reboot_app()` with confirmation dialog
EXIT TO MACOS	`QApplication.quit()`	`_shutdown_app()` with confirmation dialog

---

2. Feature Comparison

2.1 Requested Features vs. Current State

#	Requested Feature	Status	Details
1	Settings dropdown for “LED Module” vs “LCD Monitor” backend selection	❌ **MISSING**	No display backend selector exists. The system always launches the LED Simulator. There is no concept of alternative display backends.
2	Auto-boot to clock mode	❌ **MISSING**	The system always boots to the full dual-window layout (Operator Console + LED Simulator) with the LIVE tab active. There is no setting to boot directly into a clock-only/display-only mode.
3	Exit to macOS button	✅ **EXISTS**	Already implemented in `_setup_settings_tab()` at line 215-218. Red button labeled “EXIT TO MACOS” calls `_shutdown_app()` which does `QApplication.quit()` with a confirmation dialog.

2.2 What Already Exists That Supports the Requested Features

- **Settings infrastructure** is fully operational — `save_setting()` / `get_setting()` with SQLite key/value store. Adding new settings is trivial.
- **Restart mechanism** already exists (`_reboot_app()` via `os.execv`) — useful for applying backend changes.
- **Display size dropdown** already exists as a pattern — the "LED Module vs LCD Monitor" dropdown would follow the same `QComboBox` pattern.
- **Signal architecture** is clean — `model_updated` and `brightness_changed` signals mean any display backend just needs to implement `update_model(model)` and `set_brightness(value)` .

---

3. Detailed Code Analysis

3.1 `windows (1).py` — Settings/UI Elements

303 lines. Well-structured single-class `OperatorWindow(QMainWindow)` .

Existing UI elements in Settings tab:
- `self.edit_comp_name` — QLineEdit for competition name
- `self.combo_disp_size` — QComboBox for display resolution (128×64 or 64×64)
- `self.edit_led_scale` — QLineEdit for simulator LED dot scale
- `self.brightness_slider` — QSlider for display intensity (10-100%)
- `self.brightness_label` — QLabel with environmental preset names (Night/Overcast/Normal/Bright Day/Full Sun)
- **RESTART ATLAS** button — restarts the Python process
- **EXIT TO MACOS** button — graceful shutdown

What's NOT in the Settings tab:
- No "Display Backend" or "Output Device" selector
- No "Auto-boot" toggle or checkbox
- No network/connectivity settings
- No about/version info

Code quality notes:
- Methods `_reboot_app()` and `_shutdown_app()` are defined BEFORE `__init__()` (lines 11-27), which is unusual but valid Python.
- The `closeEvent` method (line 30-33) has a comment about sending a clear-screen command to LED hardware but no actual implementation beyond a print statement.
- `_save_settings()` shows a "restart required" message — this is the correct pattern for hardware config changes.

3.2 `main (1).py` — Startup Flow

45 lines. Minimal and clean.

- `DisplayConfig` is a simple data class that reads from DB on construction — it's not reactive (changes require restart).
- Both windows are created unconditionally — there's no branching logic for different startup modes.
- The LED simulator is always instantiated with `AtlasLEDSimulator(config)` .
- Signal connections are hardwired: `engine.model_updated → led_sim.update_model` .

What would need to change for auto-boot: The startup sequence would need to check a DB setting (e.g., `auto_boot_clock`) and conditionally skip showing the Operator Console, or show it minimized, or delay its appearance.

What would need to change for backend selection: Instead of always creating `AtlasLEDSimulator`, `main.py` would need to read a `display_backend` setting and instantiate the appropriate renderer. Both renderers need to implement the same interface: `update_model(model)` and `set_brightness(value)`.

3.3 `led_simulator (1).py` — Display Rendering

179 lines. Pure QPainter-based LED matrix emulation.

Current rendering pipeline:

```
engine tick (1Hz)
  → AtlasDisplayModel updated
    → model_updated signal emitted
      → AtlasLEDSimulator.update_model(model) called
        → self.update() triggers Qt repaint
          → paintEvent() → _paint_full() or _paint_compact()
            → _draw_string() → _draw_char() per character
              → QPainter.drawEllipse() per illuminated pixel (5×7 bitmap)
```

Key implementation details:
- The simulator renders **round dots** (`drawEllipse`) to simulate individual LEDs — this is specific to P10 LED panel emulation
- Colors come from the model (hex strings like `#00FFFF`, `#FFFF00`, etc.)
- Brightness is applied by scaling RGB values before painting
- Window title shows resolution: `"ATLAS VIRTUAL LED [128x64]"`
- Fixed window size — set once at construction, not resizable

For an "LCD Monitor" backend: A new class would replace the ellipse-based pixel rendering with standard Qt text rendering (QFont/QPainter.drawText), modern antialiased graphics, and a full-screen or window-sized display. It would receive the same `AtlasDisplayModel` via the same signal but render it as clean, high-resolution text rather than simulated LED dots.

3.4 `fonts.py` — Bitmap Font Capabilities

46 lines. Single class `AtlasBitmapFont` with one font: `FONT_5x7`.

Character coverage:
- Digits: 0-9 ✅
- Uppercase letters: A-Z ✅ (note: `'X'` is missing — this won't crash because `_draw_char` falls back to `' '` for unknown characters)
- Symbols: `:`, ` ` (space), `-` ✅
- Lowercase: ❌ not supported (all input is `.upper()`'d in `_draw_char`)

Bug: The letter `'Y'` is defined **twice** (lines 42 and 44). The second definition (line 44: `[0x03, 0x04, 0x78, 0x04, 0x03]`) overwrites the first. This is likely an oversight from development — one was meant to be `'X'`.

Relevance to new features: The bitmap font is only used by the LED simulator. An LCD Monitor backend would use system fonts via Qt, so this module would be untouched.

3.5 `config (1).py` — JSON Hardware Config

36 lines. Reads from `atlas_config.json`.

Current state: This module appears to be **partially obsoleted**. The `DisplayConfig` class in `main.py` reads display dimensions from the **database**, not from this JSON config. The JSON config (`atlas_config.json`) defines panel topology (`panels_wide`, `panels_high`, `double_sided`, `led_scale`) but is not referenced in `main.py`.

The `AtlasConfig` class computes display dimensions from panel counts:
- `display_width = panels_wide × 32` (e.g., 2 × 32 = 64)
- `display_height = panels_high × 16` (e.g., 4 × 16 = 64)

Note: These computed values (64×64) differ from the DB defaults used by `DisplayConfig` in main.py (128×64). This suggests the JSON config path was the original approach, later superseded by the database settings in v1.4.0.

3.6 `database (1).py` — Persistence Layer

88 lines. Clean, functional SQLite wrapper.

Two tables:
1. `heats` — heat schedule (id, heat_number, thermalling_dir, track_open/close, heat_end, status)
2. `settings` — key/value store for all application settings

Settings API: `save_setting(key, value)` and `get_setting(key, default)` — this is the insertion point for any new features. Adding a display backend preference or auto-boot flag is simply:

```
db.save_setting("display_backend", "led_simulator")
db.save_setting("auto_boot_clock", "false")
```

3.7 `timers (1).py` — Countdown Timer

41 lines. Standalone `AtlasTimerEngine` with 10Hz precision.

Current usage: This module is **NOT imported or used** by any other module in the current codebase. The engine (`engine.py`) handles its own timing via a 1Hz `QTimer` and calculates count-downs from Unix timestamps directly. This timer module appears to be a legacy component from an earlier architecture (pre-v0.8.0 “stateless timing engine” refactor).

3.8 `atlas_config.json`

```
{
    "display": {
        "panels_wide": 2,
        "panels_high": 4,
        "double_sided": false,
        "led_scale": 10
    }
}
```

Defines hardware topology. Not actively used in the current startup path (see §3.5).

3.9 `atlas.db`

The uploaded database file appears to be empty (no settings or heats rows). This is expected — it would be populated at runtime on the user's Mac Mini.

---

4. Gap Analysis & Recommendations

4.1 Feature 1: Display Backend Selector ("LED Module" vs "LCD Monitor")

What exists: Nothing. The system is hardwired to `AtlasLEDSimulator`.

What needs to be added:

1. **New setting in Settings tab** — Add a `QComboBox` labeled "Display Output" with options:
   - `"LED Module (P10 Simulator)"` — current behavior
   - `"LCD Monitor (Full Screen)"` — new renderer

Insert it into `_setup_settings_tab()` alongside the existing display size combo. Save to DB as `display_backend`.

1. **New LCD renderer class** — Create a new file (e.g., `gui/lcd_display.py`) implementing a `QWidget` with:
   - Same interface as `AtlasLEDSimulator`: `update_model(model)` and `set_brightness(value)` methods
   - Standard Qt text rendering (QFont, QPainter.drawText) instead of bitmap pixel art
   - Full-screen or large-window mode
   - Clean, antialiased display suitable for monitors/TVs

2. **Conditional instantiation in `main.py`** — Read `display_backend` setting and create the appropriate renderer:
   ```
   python
   backend = db.get_setting("display_backend", "led_simulator")
   if backend == "lcd_monitor":
       display = AtlasLCDDisplay(config)
   else:
       display = AtlasLEDSimulator(config)
   engine.model_updated.connect(display.update_model)
   engine.brightness_changed.connect(display.set_brightness)
   ```

3. **Save & restart flow** — Already exists. The "SAVE & APPLY" button + restart message + RESTART ATLAS button handle this.

Risk: LOW. The signal architecture already decouples the engine from the display. Adding a second renderer doesn't touch any existing rendering code.

4.2 Feature 2: Auto-Boot to Clock Mode

What exists: Nothing. Both windows always show.

What needs to be added:

1. **New setting in Settings tab** — Add a `QCheckBox` labeled "Auto-boot to Clock Mode" (or similar). Save to DB as `auto_boot_clock`.

2. **Startup logic in `main.py`** — Check the setting after creating both windows:
   ```
   python
      auto_boot = db.get_setting("auto_boot_clock", "false") == "true"
      if auto_boot:
          # Show only the display window; hide or minimize operator console
          operator_win.hide()  # or operator_win.showMinimized()
      else:
          operator_win.show()
      display.show()
   ```
3. **Escape hatch** — Need a way to access the Operator Console when in auto-boot mode. Options:
   - Keyboard shortcut (e.g., `Ctrl+O` or `Escape`) to toggle Operator Console visibility
   - Right-click context menu on the display window
   - System tray icon

Risk: LOW. This is purely additive startup logic. The core system is unaffected.

Consideration: If the system is meant to run as a kiosk (always showing the clock), the display window should probably also be set to `showFullScreen()` in auto-boot mode.

4.3 Feature 3: Exit to macOS Button

Status: ✅ **ALREADY EXISTS.** No changes needed.

- **Location:** `windows.py` line 215-218
- **Label:** "EXIT TO MACOS"
- **Style:** Red button (`background: #900`)
- **Behavior:** Confirmation dialog → `QApplication.quit()`
- **Companion:** "RESTART ATLAS" button alongside it

4.4 Summary: What Stays, What Changes

Component	Action	Rationale
`engine.py`	**NO CHANGE**	Signal architecture already supports multiple display backends
`models.py`	**NO CHANGE**	`AtlasDisplayModel` is backend-agnostic
`database.py`	**NO CHANGE**	`save_setting` / `get_setting` already handles arbitrary keys
`timers.py`	**NO CHANGE**	Not currently used; leave as-is
`fonts.py`	**NO CHANGE**	Only used by LED simulator; LCD backend uses Qt fonts
`themes.py`	**NO CHANGE**	Operator Console styling only
`config.py`	**NO CHANGE**	Legacy module; not in the critical path
`atlas_config.json`	**NO CHANGE**	Legacy config; not in the critical path
`windows.py`	**MINOR ADDITION**	Add ~20 lines: display backend dropdown + auto-boot checkbox in `_setup_settings_tab()`, persist both in `_save_settings()`
`main.py`	**MINOR MODIFICATION**	Add ~15 lines: read backend setting → conditional display instantiation; read auto-boot → conditional window visibility
NEW: `gui/lcd_display.py`	**NEW FILE**	New LCD monitor renderer class (~100-150 lines)

4.5 Risks & Compatibility Concerns

Risk	Severity	Mitigation
Breaking existing LED simulator behavior	LOW	LED simulator code is completely untouched; it remains the default backend
Display config mismatch on first run	LOW	Use `db.get_setting("display_backend", "led_simulator")` — defaults to current behavior if no setting exists
Auto-boot with no escape hatch	MEDIUM	Must implement a keyboard shortcut or other mechanism to access Operator Console when it's hidden
`config.py` vs `database.py` settings confusion	LOW	`config.py` is not in the critical path; document that DB settings are authoritative
Duplicate `_draw_char` in `led_simulator.py`	COSMETIC	The second (brightness-aware) definition wins at runtime; first is dead code. Can clean up but not urgent.
Missing `'X'` glyph in fonts.py	COSMETIC	Falls back to blank space. Only affects LED simulator. Fix: rename duplicate `'Y'` to `'X'`.
`timers.py` is unused	COSMETIC	Orphaned module from pre-v0.8.0 architecture. No impact.

4.6 Recommended Implementation Order

1. **Add settings UI** (dropdown + checkbox) to `windows.py` — fast, no risk, immediately testable
2. **Add conditional startup** logic to `main.py` — read new settings, branch on backend and auto-boot
3. **Build LCD display class** — new file, implements same interface as LED simulator
4. **Test round-trip** — change setting → restart → correct display backend launches
5. **Add auto-boot escape hatch** — keyboard shortcut to toggle Operator Console

Total estimated surgical changes: ~3 files modified, ~1 new file created, ~50 lines of modifications to existing code.

Appendix A: Git History Summary

The HEAD reflog shows 27 commits from initial setup to v1.6.0, tracking a methodical development process with version numbers and change descriptions. Key milestones:

Version	Milestone
v0.1.0	Skeleton baseline, dual-window architecture
v0.4.0	Timeline engine, auto-transitioning phases
v0.5.0	SQLite persistence layer
v0.6.0	LED hardware-aligned layout
v0.8.0	Single Source of Truth architecture, stateless timing engine
v0.9.0	128×64 P10 Virtual LED Matrix with bitmap font renderer
v1.0.0	Configuration abstraction layer (JSON config)
v1.2.0	Custom Nav-Bar (retired native TabWidget)
v1.3.0	Display-aware rendering (compact mode for 64×64)
v1.4.0	Unified configuration — settings migrated to SQLite
v1.5.0	Live brightness control
v1.6.0	Appliance controls (restart/exit)

Appendix B: File Metrics

File	Lines	Purpose
`main (1).py`	45	Application entry point
`engine (1).py`	91	State engine + heat phase logic
`database (1).py`	88	SQLite persistence
`models (1).py`	24	Pydantic data models
`config (1).py`	36	JSON hardware config (legacy)
`timers (1).py`	41	Standalone countdown timer (unused)
`windows (1).py`	303	Operator Console GUI
`led_simulator (1).py`	179	LED matrix renderer
`themes (1).py`	49	CSS stylesheets
`fonts.py`	46	5×7 bitmap font data
Total	**902**	